

Session 3.5

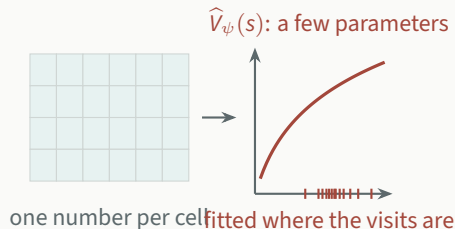
From Tables to Networks: Function Approximation, Policy Gradient, Actor–Critic

Parameters replace the table; then a gradient step replaces the max



Zhigang Feng

Where a Table Stops Fitting, and What Deck 3.2's Critic Already Did



- ▶ Deck 3.4 kept a table of 68, one number per feasible cell. Deck 3.2's critic never had cells: $\hat{V}_\psi = \psi^\top \phi$, **five numbers**.
- ▶ Capital is continuous; with heterogeneous agents the state holds a distribution.
- ▶ **Function approximation**: far fewer parameters than states — fitted where the samples fall, not at nodes you placed. A table is its indicator-feature case.
- ▶ **Two routes: a basis an economist picks, then features a network learns. Then the max goes too.**

Source: Miao & Feng, *AI for Economists*, ch. 12; K. Judd, *Numerical Methods in Economics*, 1998, ch. 11.

Session 3.5

Route One: Linear in Parameters

A basis the economist picks — where the theory still holds, and what it promises



Linear Value Functions and Semi-Gradient TD

- Pick p **basis functions**: $\hat{V}_\psi(s) = \psi^\top \phi(s)$, linear in ψ , not in the state. With $\tilde{k}$ capital rescaled to $[0, 1]$:

$$\phi(k, z) = (1, \tilde{k}, \tilde{k}^2, \mathbf{1}\{z = 1.1\}, \tilde{k} \cdot \mathbf{1}\{z = 1.1\})^\top.$$

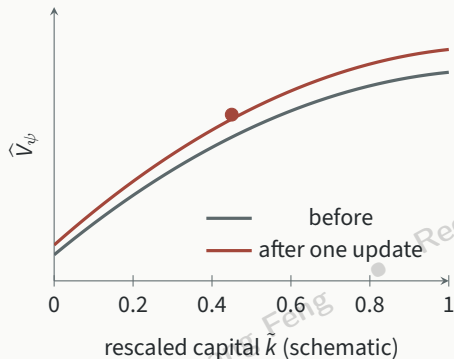
- The TD error $\delta_t = r_{t+1} + \gamma \hat{V}_\psi(s_{t+1}) - \hat{V}_\psi(s_t)$ is unchanged. There is no cell to correct, so move ψ :

$$\psi \leftarrow \psi + \eta \delta_t \nabla_\psi \hat{V}_\psi(s_t) = \psi + \eta \delta_t \phi(s_t).$$

- **Semi-gradient**: the target depends on ψ too, but is held constant (detached, in PyTorch).
- **A table is this with indicator features, one per cell: then $\phi(s_t)$ selects one entry and nothing else moves.**

Source: R. Sutton, A. Barto, *Reinforcement Learning*, 2018, §9.3; Miao & Feng, *AI for Economists*, ch. 12.

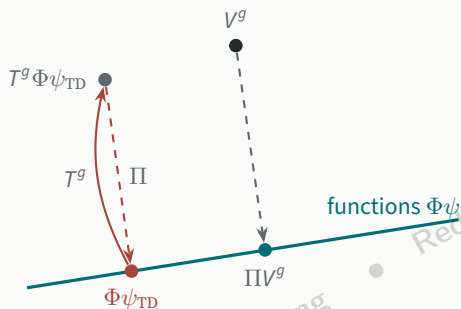
One Update Moves Many States



- ▶ A table corrects one cell. A step on ψ shifts every state that shares $\tilde{k}$ and $\tilde{k}^2$.
- ▶ Gain: one visit updates a neighbourhood; unvisited states get an estimate.
- ▶ Risk: a poor basis pushes error onto states that did nothing wrong.
- ▶ **Generalisation is the point, and the source of the failures.**

Source: Miao & Feng, *AI for Economists*, ch. 12, “Linear approximation and semi-gradient TD”. Curves are schematic.

Where Linear TD Converges: the Projected Bellman Equation



- ▶ On-policy data, full-rank features: ψ converges to the **TD fixed point** $\Phi\psi = \Pi T^g \Phi\psi$.
- ▶ Π : least squares weighted by the stationary distribution $\bar{d}_g$.
- ▶ Not the best fit, but within $\frac{1}{1-\gamma}$ of it in the $\bar{d}$ -norm.
- ▶ **At $\gamma = 0.95$ that factor is 20; a sharper argument gives 3.2. Off-policy data void the theorem.**

Source: J. Tsitsiklis, B. Van Roy, *IEEE Trans. Automatic Control* 42(5), 1997, Theorem 1; $(1 - 0.95^2)^{-1/2} = 3.2$.

The TD Fixed Point Is an Instrumental-Variables Estimator

- ▶ The fixed point is a moment condition under the stationary distribution:

$$E\left[\phi(s_t)(r_{t+1} + \gamma\phi(s_{t+1}))^\top \psi - \phi(s_t)^\top \psi\right] = 0.$$

- ▶ As a regression, $r_{t+1} = (\phi(s_t) - \gamma\phi(s_{t+1}))^\top \psi + e_{t+1}$: the regressor contains s_{t+1} , whose randomness is also in e_{t+1} . OLS is biased; $\phi(s_t)$, fixed beforehand, is a valid instrument.
- ▶ Solving the sample moments gives **least-squares TD** (LSTD):

$$\hat{\psi}_{\text{LSTD}} = \left[\sum_t \phi(s_t)(\phi(s_t) - \gamma\phi(s_{t+1}))^\top \right]^{-1} \sum_t \phi(s_t) r_{t+1}.$$

- ▶ **Semi-gradient TD walks towards this solution step by step. LSTD solves for it at once.**

Source: S. Bradtke, A. Barto, *Machine Learning* 22, 1996; J. Boyan, *Machine Learning* 49, 2002; Miao & Feng, ch. 12.

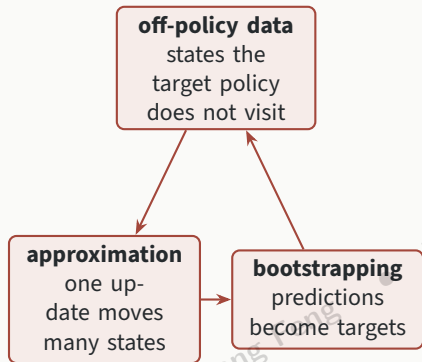
Measured: Three Linear Bases, One Policy

basis	weighted error		max error	bound	
	best fit	TD point	TD point	theorem	sharper
5 terms, polynomial in $\tilde{k}$	0.025	0.027	0.902	0.493	0.079
4 terms, in $\ln k$	0.023	0.023	0.066	0.459	0.074
3 terms, no curvature	0.144	0.165	1.738	2.887	0.462

- ▶ Recorded on the 7×2 grid, evaluating Deck 3.4's ϵ -greedy rule: errors in value units, not consumption.
- ▶ Rows 1 and 2: same weighted error, maximum error 14 times apart. With log utility, economics says use $\ln k$.
- ▶ **On 200,000 steps LSTD errs by 0.030; semi-gradient TD, at the same point, is at 0.190.**
- ▶ *Continuous model*: Deck 3.2's five-feature critic ends at rms 1.72 against V^* — a poor fit, on a rule that still scores 0.014%. **A critic is a means, not an end.**

Source: The book's script for ch. 12, 7×2 grid, errors weighted by $\bar{d}$; $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2. Continuous model: Session 3 lab, block A2.

The Deadly Triad



- ▶ Function approximation, bootstrapping and off-policy sampling: any two are safe, all three can diverge. Baird's star is the standard counter-example.
- ▶ Linear TD on the rule's own data has only the first two — which is why Deck 3.2's critic has a guarantee at all. **Deck 3.2 is not in the triad:** a linear critic, on its own path.
- ▶ **With a neural critic there is no general global convergence result**, and the book says so of its own chapter's algorithms.

Source: L. Baird, "Residual algorithms", *ICML*, 1995; Sutton & Barto, 2018, §11.3; Miao & Feng, ch. 12, §12.6 footnote.

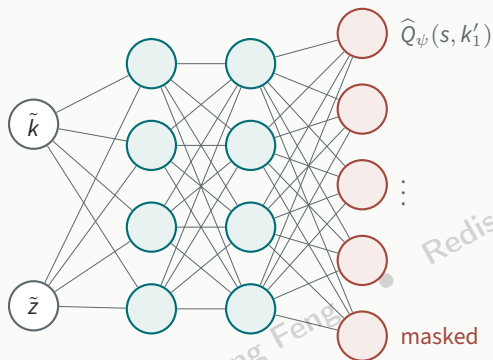
Session 3.5

Route Two: Learn the Features

A network for Q , then the policy itself — and then stop taking the max



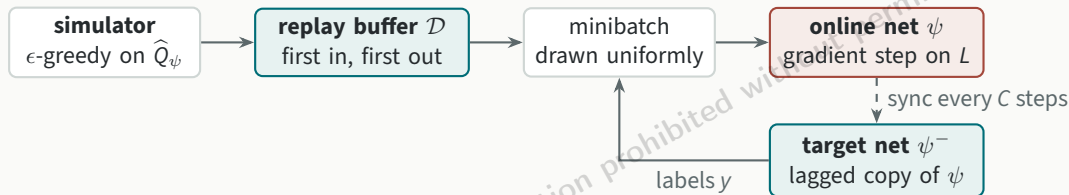
Let the Features Be Learned: a Network for Q



- ▶ A linear basis is as good as the economist's guess. A network's early layers *are* the basis, and they are trained.
- ▶ One output per action k' ; infeasible ones are masked before the max.
- ▶ In Q-learning this completes the triad: it oscillates or diverges.
- ▶ **DQN adds two devices that hold the regression still.**

Source: V. Mnih et al., "Human-level control through deep reinforcement learning", *Nature* 518, 2015.

DQN: a Target Network and a Replay Buffer

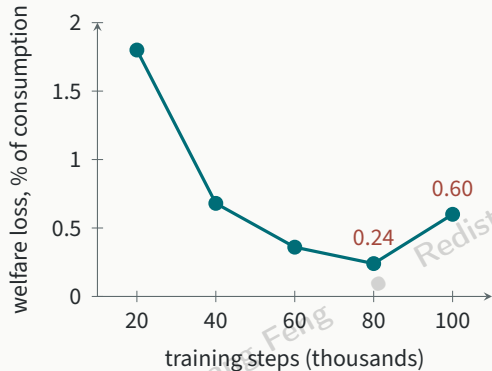


$$y_t = r_{t+1} + \gamma \max_{a' \in \mathcal{A}(s_{t+1})} \hat{Q}_{\psi^-}(s_{t+1}, a'), \quad L(\psi) = E_{\mathcal{D}} [(y - \hat{Q}_\psi(s, a))^2].$$

- **Target network:** a frozen label, so the regression has a fixed target. It acts on bootstrapping.
- **Experience replay:** decorrelates successive samples and reuses them. It does not repair the off-policy mismatch.
- **Neither removes the triad. They make training stable enough, with tuning.**

Source: Mnih et al., *Nature* 518, 2015; Miao & Feng, *AI for Economists*, ch. 12, “Neural networks and DQN”.

DQN on the Growth Model, Recorded



- ▶ A network for $\hat{Q}$ on the 20×2 grid, 40 states: two hidden layers of 64 units, sync every 500 gradient steps, ϵ from 1 to 0.1.
- ▶ At 100,000 steps the greedy rule matches dynamic programming in 26 of the 40 states; welfare loss 0.60%.
- ▶ Random actions lose over 100%. The book's three seeds: 0.60, 0.38, 0.20%.
- ▶ **The best iterate came at 80,000 steps, not at the end.**

Source: Recorded by the book's script for ch. 12; $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2; 20×2 grid, seed 0; consumption-equivalent loss, visit-weighted.

Parameterise the Policy: a Share, or a Masked Softmax

- ▶ Double DQN and Rainbow refine that max; none removes it. A portfolio share cannot be enumerated, and $\arg \max_a \hat{Q}_\psi$ flips on a small change in ψ .
- ▶ So make the policy differentiable in θ . For discrete actions, a **masked softmax**:

$$g_\theta(a | s) = \frac{\exp h_\theta(s, a)}{\sum_{a' \in \mathcal{A}(s)} \exp h_\theta(s, a')}, \quad a \in \mathcal{A}(s),$$

zero elsewhere — **the 7×2 grid's parameterisation.**

- ▶ **For a real-valued action, put the density on the action:** Deck 3.2 draws $x \sim N(\mu_\theta, \sigma^2)$ and sets $a = \text{sigmoid}(x)$. **The theorem below covers both.**
- ▶ **Q-learning fits $\hat{Q}$ to Bellman's equation, then maximises. This maximises V^{g_θ} directly.**

Source: H. van Hasselt, A. Guez, D. Silver, AAAI, 2016; M. Hessel et al., AAAI, 2018; Miao & Feng, ch. 12, “Improving the policy directly”.

The Policy-Gradient Theorem

- ▶ If g_θ is differentiable in θ , rewards are bounded and $\gamma < 1$, then with G_t the return from t :

$$\nabla_\theta J(\theta) = E \left[\sum_{t \geq 0} \gamma^t \nabla_\theta \ln g_\theta(a_t | s_t) G_t \right] = \frac{1}{1 - \gamma} \sum_{s,a} d_{g_\theta}(s, a) \nabla_\theta \ln g_\theta(a | s) Q^{g_\theta}(s, a).$$

- ▶ $\nabla_\theta \ln g_\theta$ is the **score** of maximum likelihood: it makes a_t more likely in s_t . The return is its weight.
- ▶ First form: estimate the gradient from simulated paths. Second form: Q^{g_θ} is still inside, weighted by the **discounted visit distribution** d_{g_θ} that the γ^t builds.
- ▶ **This is the debt Deck 3.2 recorded** — row four of *Nine Numbers, Six Ideas*: improve by a gradient, not a max. **Algorithms differ only in how they estimate Q^{g_θ} : whole returns now, a TD critic later.**

Source: Sutton, McAllester, Singh & Mansour, *NeurIPS* 12, 2000; Marbach & Tsitsiklis, *IEEE TAC* 46(2), 2001.

Why the Environment Drops Out

$$P_{\theta}(\tau) = P_0(s_0) \underbrace{g_{\theta}(a_0 | s_0)}_{\text{contains } \theta} \underbrace{P(s_1 | s_0, a_0)}_{\text{does not}} \underbrace{g_{\theta}(a_1 | s_1)}_{\text{contains } \theta} P(s_2 | s_1, a_1) \dots$$

- ▶ Deck 3.2 drew this factorisation to explain why no P_z appeared in its seven lines. Here is what it proves.
- ▶ Differentiate $J = \int P_{\theta}(\tau) R(\tau) d\tau$ with $\nabla P_{\theta} = P_{\theta} \nabla \ln P_{\theta}$. In $\ln P_{\theta}(\tau)$ the simulator's factors have zero derivative: $\nabla_{\theta} \ln P_{\theta}(\tau) = \sum_t \nabla_{\theta} \ln g_{\theta}(a_t | s_t)$.
- ▶ No transition probability is needed: simulated paths are enough. Rewards earned before a_t drop out in expectation.
- ▶ **Keep the γ^t : dropping it biases the update — 10.2% in the book's network run, though the recorded tabular run's line 6 drops it and survives.**

Source: Miao & Feng, *AI for Economists*, ch. 12, "The policy-gradient theorem"; P. Thomas, *ICML*, 2014.

REINFORCE with a Baseline

REINFORCE with a learned baseline $\hat{V}_\psi$

- 1 initialise θ, ψ ; **repeat** until the iteration limit:
- 2 draw $s_0 \sim P_0$; simulate T periods under g_θ : $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$
- 3 backwards: $G_{t:T} = r_{t+1} + \gamma G_{t+1:T}$, with $G_{T:T} = 0$
- 4 $\Delta\theta \leftarrow \sum_{t < T_1} \gamma^t \nabla_\theta \ln g_\theta(a_t | s_t) (G_{t:T} - \hat{V}_\psi(s_t))$
- 5 $\Delta\psi \leftarrow \sum_{t < T_1} (G_{t:T} - \hat{V}_\psi(s_t)) \nabla_\psi \hat{V}_\psi(s_t)$
- 6 $\theta \leftarrow \theta + \eta_\theta \Delta\theta, \quad \psi \leftarrow \psi + \eta_\psi \Delta\psi$
 - A **baseline** $v(s)$ is free: $E_a[\nabla_\theta \ln g_\theta v(s)] = v(s) \nabla_\theta \sum_a g_\theta(a | s) = 0$. With $v = V^g$ the weight is the **advantage** $A^g = Q^g - V^g$.
 - Lines 4–5 sum only over $t < T_1$: with $T = 300$ -period paths, only the first $T_1 = 150$ returns are used, so every one of them has at least 150 real rewards behind it.
 - **Line 4 is Monte Carlo evaluation again: whole returns, used once, after the path ends.**

Source: R. Williams, *Machine Learning* 8, 1992; Miao & Feng, *AI for Economists*, ch. 12, “REINFORCE and baselines”.

Measured: a Baseline Is a Control Variate

baseline	summed variance		mean of 400 paths	
	value	relative	norm	distance to exact
none	141.8	1	1.378	0.618
the true value V^{g_θ}	17.3	0.122	1.489	0.228
a learned $\hat{V}$	17.4	0.123	1.492	0.229

- ▶ A uniform random policy on the 7×2 grid; exact gradient norm 1.420; one 300-period path per estimate.
- ▶ The mean is unchanged within sampling error; the variance falls to an eighth.
- ▶ **A rough baseline does as well as the true one. A control variate needs correlation, not accuracy.**

Source: Recorded by the book's script for ch. 12; $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2; 7×2 grid. Greensmith, Bartlett & Baxter, *JMLR* 5, 2004.

Session 3.5

Actor-Critic

Put evaluation back inside improvement — and settle Deck 3.2's account



What REINFORCE Still Pays, and the One Change That Fixes It

REINFORCE: weight $G_{t:T} - v(s_t)$

Actor-critic: weight δ_t

waits for the path to end

δ_t is known one period later

cuts the infinite horizon at T

$\gamma \hat{V}_\psi(s_{t+1})$ carries the whole tail

carries the noise of a whole path

one reward, one transition

- ▶ With $\delta_t = r_{t+1} + \gamma \hat{V}_\psi(s_{t+1}) - \hat{V}_\psi(s_t)$ and an exact critic, $E[\delta_t \mid s_t, a_t] = A^g(s_t, a_t)$.
- ▶ This is Deck 3.2's sample backup (*Sampling the Tree*) with $T = 1$. An n -step return minus the baseline sits in between.
- ▶ **The price is bias: a wrong critic points the actor the wrong way.**

Source: Miao & Feng, *AI for Economists*, ch. 12, “What was gained, what was lost” and “Actor-critic”.

One-Step Actor-Critic

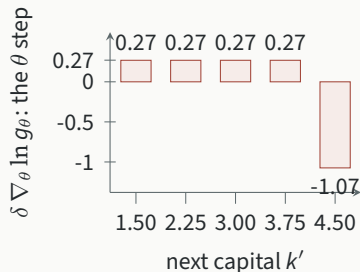
One-step actor-critic, online

- 1 initialise θ, ψ ; draw s from the start distribution
- 2 **for** $t = 0, 1, 2, \dots$ up to the step limit:
- 3 draw $a \sim g_\theta(\cdot | s)$ on the feasible set; the simulator returns r and s'
- 4 $\delta \leftarrow r + \gamma \widehat{V}_\psi(s') - \widehat{V}_\psi(s)$ (bootstrap through resets and truncation)
- 5 $\psi \leftarrow \psi + \eta_\psi \delta \nabla_\psi \widehat{V}_\psi(s)$ (critic: semi-gradient TD)
- 6 $\theta \leftarrow \theta + \eta_\theta \delta \nabla_\theta \ln g_\theta(a | s)$ (actor: policy gradient)
- 7 at a reset, redraw s ; otherwise $s \leftarrow s'$

- The **actor** is the policy; the **critic** is a value estimate trained by TD on the actor's data.
- $\delta > 0$: better than the critic expected, so a becomes more likely.
- **Evaluation and improvement take one step each, on the same transition. Deck 3.2 ran these seven lines with eight numbers and no table.**

Source: Miao & Feng, *AI for Economists*, ch. 12, "Actor-critic"; Sutton & Barto, *Reinforcement Learning*, 2018, §13.5.

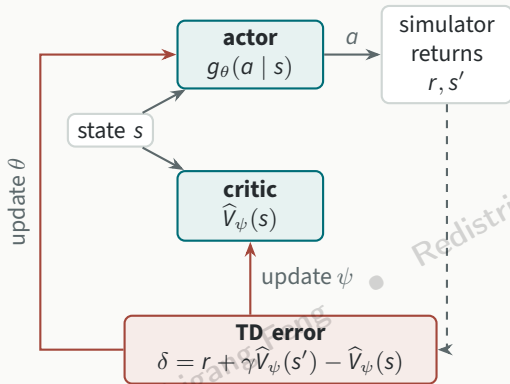
The Demo's Actor-Critic, Now Derived



- ▶ On a grid the actor is this softmax with **indicator features**: one ϕ per cell, so the score closes to $\mathbf{1}\{\cdot = a\} - g_{\theta}(\cdot | s)$ in row s , zero elsewhere.
- ▶ **The plot is the actual θ step**, score times δ : at $\delta = -1.3425$ the drawn action 4.50 falls by 1.074 and every other feasible action rises by 0.269.
- ▶ **This is the recorded 7×2 run**: the same algorithm with a 14-cell critic and a 68-weight softmax, scoring 0.52 %. Deck 3.6's ladder uses it.
- ▶ Deck 3.2's actor is the continuous cousin, with score $\frac{x-\mu}{\sigma^2} \nabla_{\theta} \mu$. **One theorem covers both.**

Source: Sutton et al., *NeurIPS* 12, 2000; A. Agarwal, S. Kakade, J. Lee, G. Mahajan, *JMLR* 22, 2021.

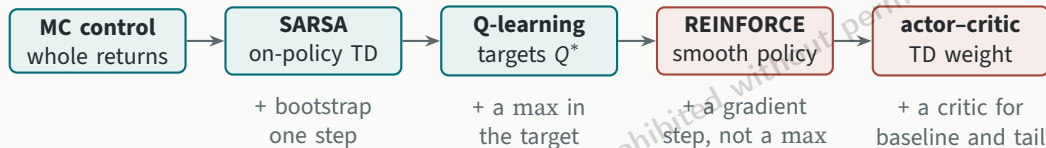
The Data Flow: One TD Error, Two Updates



- ▶ One loss in code:
 $L = \frac{1}{2}\delta^2 - \delta \ln g_{\theta}(a | s)$. Detach the target for the critic, and δ for the actor.
- ▶ The critic must keep up with a moving actor: its step size is larger.
- ▶ With a known, differentiable model the actor can take a **pathwise gradient** through the Bellman right-hand side.
- ▶ **That variant is Deck 3.6's deep value iteration.**

Source: V. Konda, J. Tsitsiklis, "Actor-critic algorithms", *NeurIPS* 12, 2000; Miao & Feng, *AI for Economists*, ch. 12.

One Ladder: from Monte Carlo Control to Actor-Critic



- ▶ Each rung changes one thing; none reads the model. Deck 3.2 climbed it backwards, and said so.
- ▶ *Tables* on the 7×2 grid, medians of 20 seeds: REINFORCE loses 1.68 %, actor-critic 0.52 %. *Networks* on the 20×2 grid, seed 0: 0.73 % and 0.19 %, with the other two seeds reversing it.
- ▶ **Two grids, two representations: the rows are not comparable. A ladder of ideas is not a ranking of results — Deck 3.6 runs one that is.**

Source: Recorded by the book's script for ch. 12; $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2; visit-weighted; network seeds 1, 2 from the chapter. Continuous model: Session 3 lab, block B.

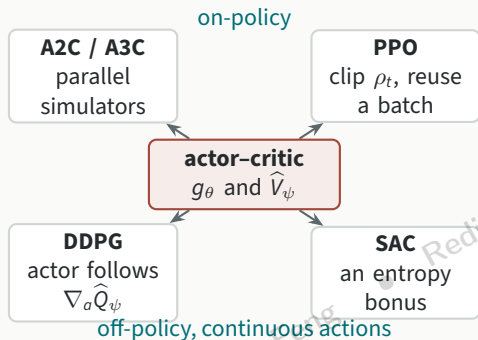
Why Economists Should Care: Actor–Critic as Two-Step Estimation

	Two-step estimation	Actor–critic
nuisance parameter	first stage	the critic — five numbers in Deck 3.2: $E[\delta_t \nabla_{\psi} \hat{V}_{\psi}(s_t)] = 0$
target parameter	second stage	the actor — three numbers: zero update at the optimum
timing	stage one, then stage two	interleaved on every sample

- ▶ Deck 3.2 ran a two-step estimator before it had the name: **eight numbers, one moment condition each**. The stages interleave, so the critic must move faster — in the worked period it moved four hundred times faster.
- ▶ **The saving rule is the estimand; the value function is the nuisance that makes it estimable.**

Source: W. Newey, D. McFadden, *Handbook of Econometrics* 4, 1994, ch. 36; Miao & Feng, ch. 12.

Four Modern Variants, Four Standing Difficulties



- ▶ All four keep an actor and a critic; only the objective or the reuse of data changes. PPO's ratio is Deck 3.3's and its advantage is Deck 3.4's λ -return; PPO also aligns language models (Session 5).
- ▶ None of them removes this deck's four bills: 10^5 steps for 40 states, seed reversals, early lock-in, a last iterate that was not the best.
- ▶ **Grade against an exact solution and report seeds as a range. Next, Deck 3.6: every rung, one model, one protocol.**

Source: Mnih et al., *ICML*, 2016; Schulman et al., 2017; Lillicrap et al., *ICLR*, 2016; Haarnoja et al., *ICML*, 2018; P. Henderson et al., *AAAI*, 2018.